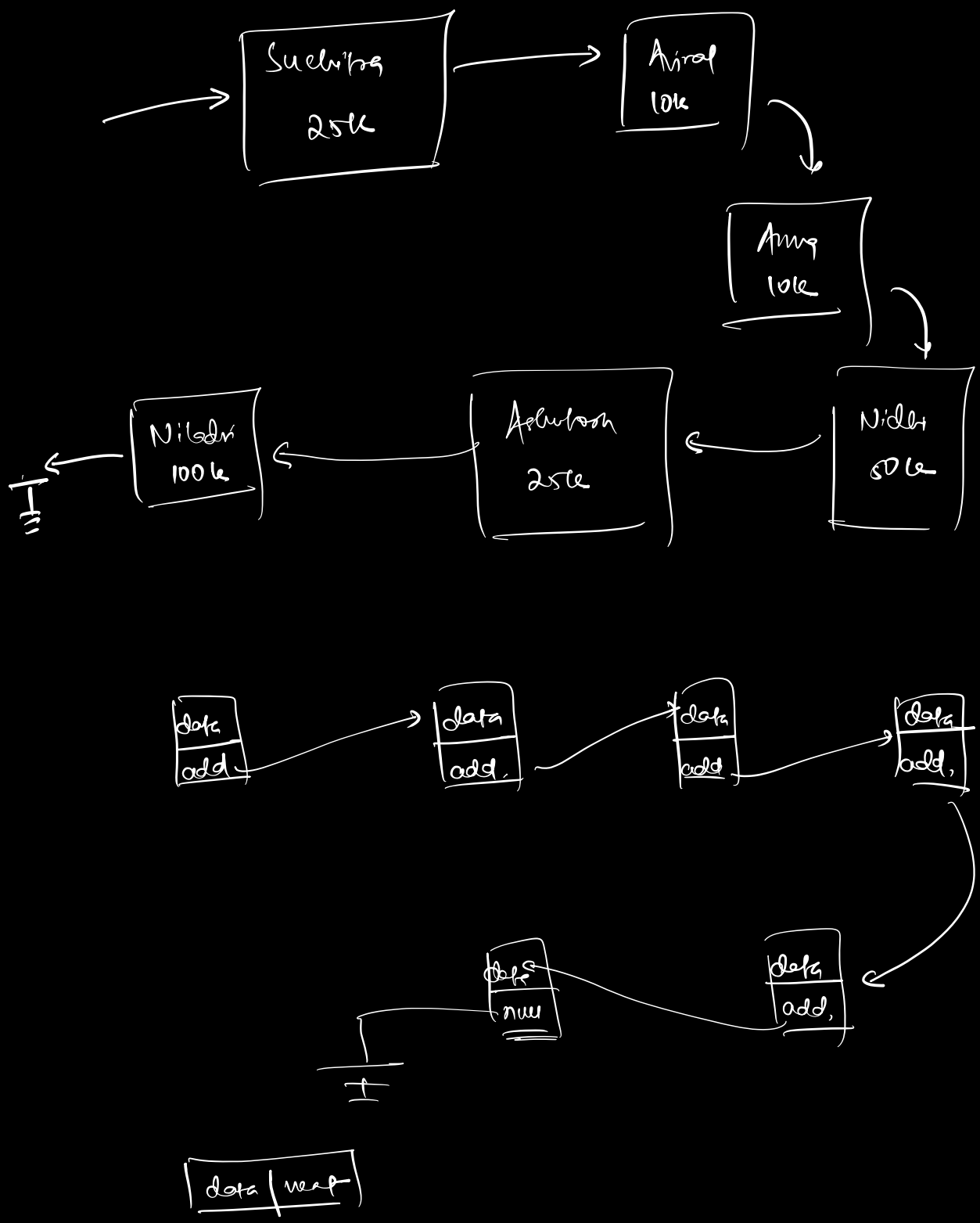
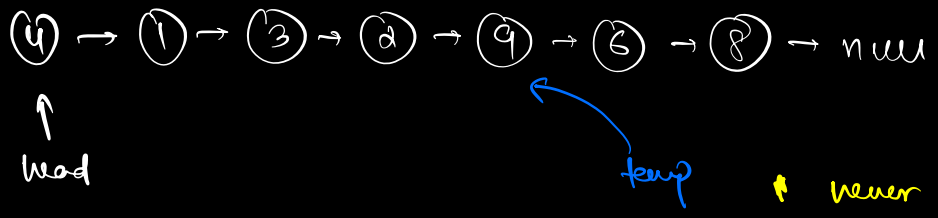

Sudhir
25k
Viral
10k
Anuj
10k
Nidhi
60k
Ashutosh
25k
Nitesh
100k
data
add
data
add
data
add
data
add
data
null
data
add
data
next

```
class Node {
    int data;
    Node next;
    Node(int x){
        data = x
        next = null
    }
```

⇒ Search an element in a LL:



k = 9

* never change your head

```
bool search(head, k) {
    Node temp = head;
    while(temp != null) {
        if(temp.data == k)
            return true
        }
        temp = temp.next
    }
    return false;
}
```

Search in a sorted LL

Can we do binary search

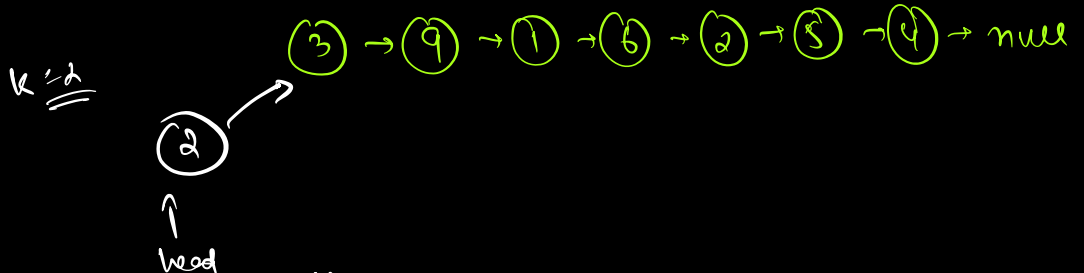
arr[mid] == k

> k

< k

⇒ Insertion in a LL :

* Insert at head :-



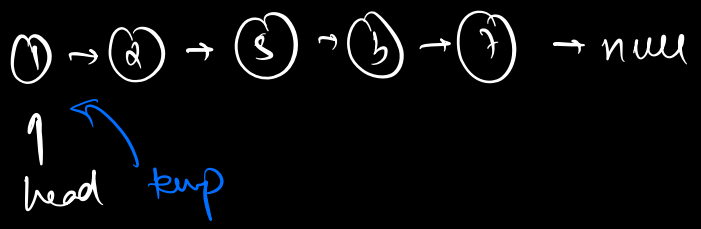
Node newNode = new Node(2);

newNode.next = head;

head = newNode;

TC ⇒ O(1)

* or tail



k = 10

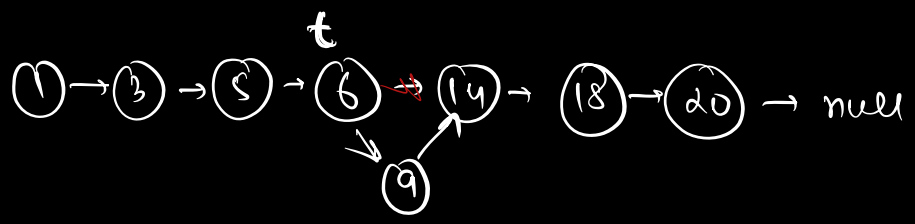
```
Node newNode = new Node(x);
if(head == null) {
        head = newNode
        return head;
}
temp = head
while(temp.next != null){
        temp = temp.next
}
   temp.next = newNode;
}
```

O(N)

O(1)


```
t.next = newNode
t = t.next
```

Sorted LL → Insert element in a Sorted LL



k = 9

```
Node newNode = new Node(k)
if( head == null ){
        head = newNode
        return
}
if( head.data > k ){
        newNode.next = head
        head = newNode
}
while( temp.next != null
    && temp.next.data ≤ k ){
        temp = temp.next
}
newNode.next = temp.next
temp.next = newNode
```

Short-circuit

||	&&
↓	
01 → 1	00 → 0
10 → 1	01 → 0
11 → 1	10 → 0
00 → 0	11 → 1

```
if(temp==null || temp.data > k ) {

}
```

```
if( cond1 && cond2 ) {
```

(cond1 → false)

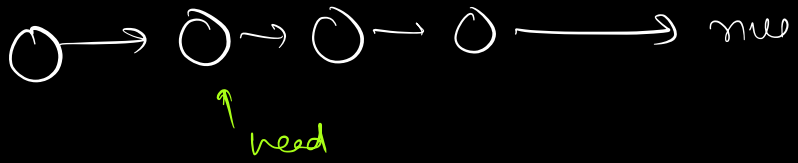
→ (&& ||) ✓

→ (& |) ✗

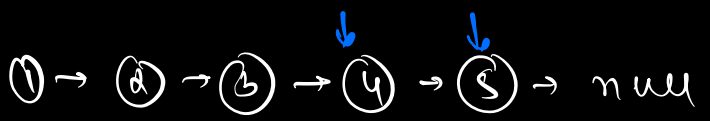
⇒ Deletion in a LL :

* from head :

head = head.next

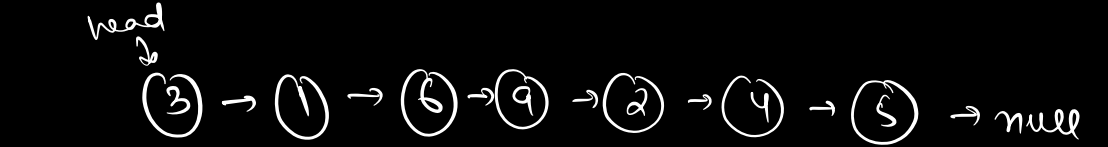


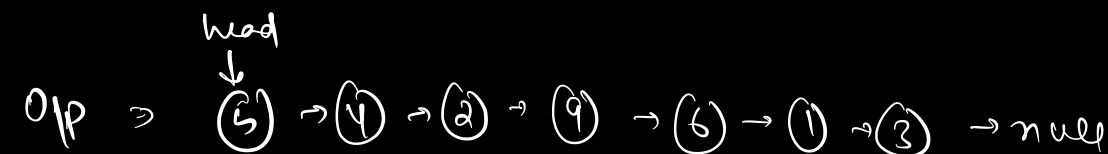
** from tail

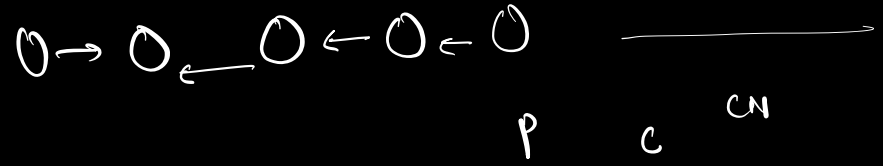



```
while( temp.next.next != null) {
        temp = temp.next;
}
temp.next = null;
```

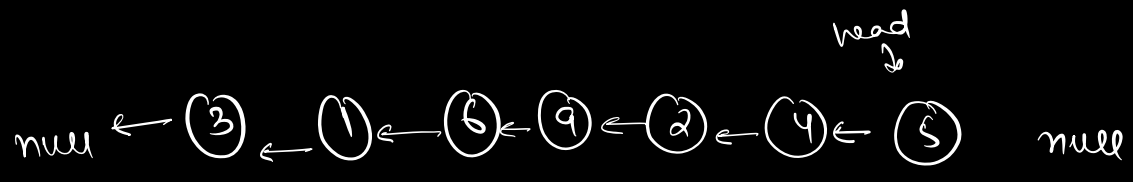
Q reverse a LL.








```
C.next = P
P = C
C = CN
CN = CN.next
```

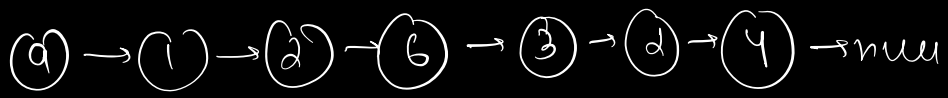



```
while(CN != null)
{
    C.next = P
    P = C
    C = CN
    if(CN != null)
    CN = CN.next
}
head.next = null
head = P
```

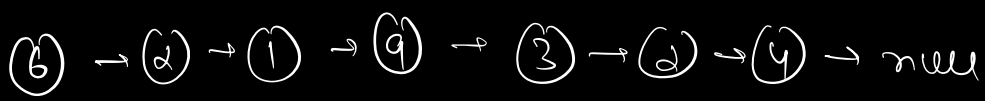
TC ⇒ O(N)

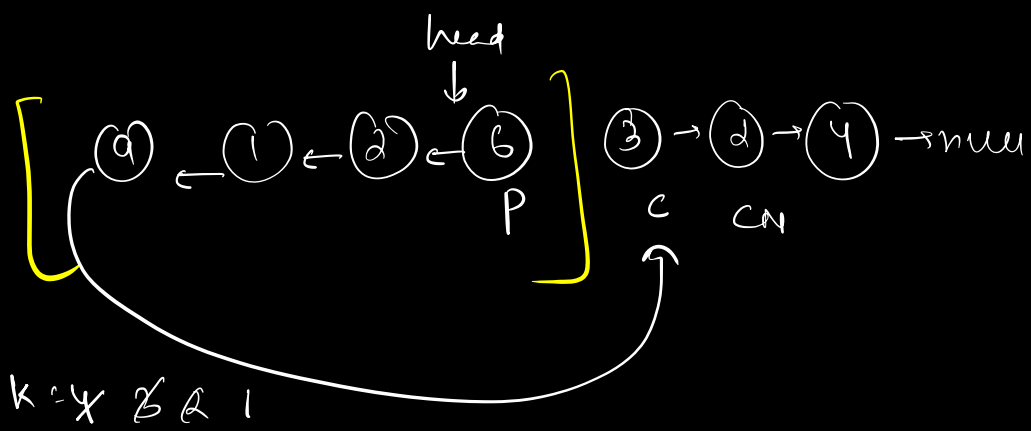
SC ⇒ O(1)

Reverse first k nodes of a LL :



k = 4





head.next = C

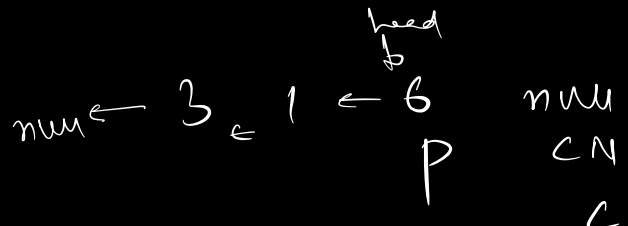
head = P

C.next = P

P = C

C = CN

if(CN != null)
CN = CN.next

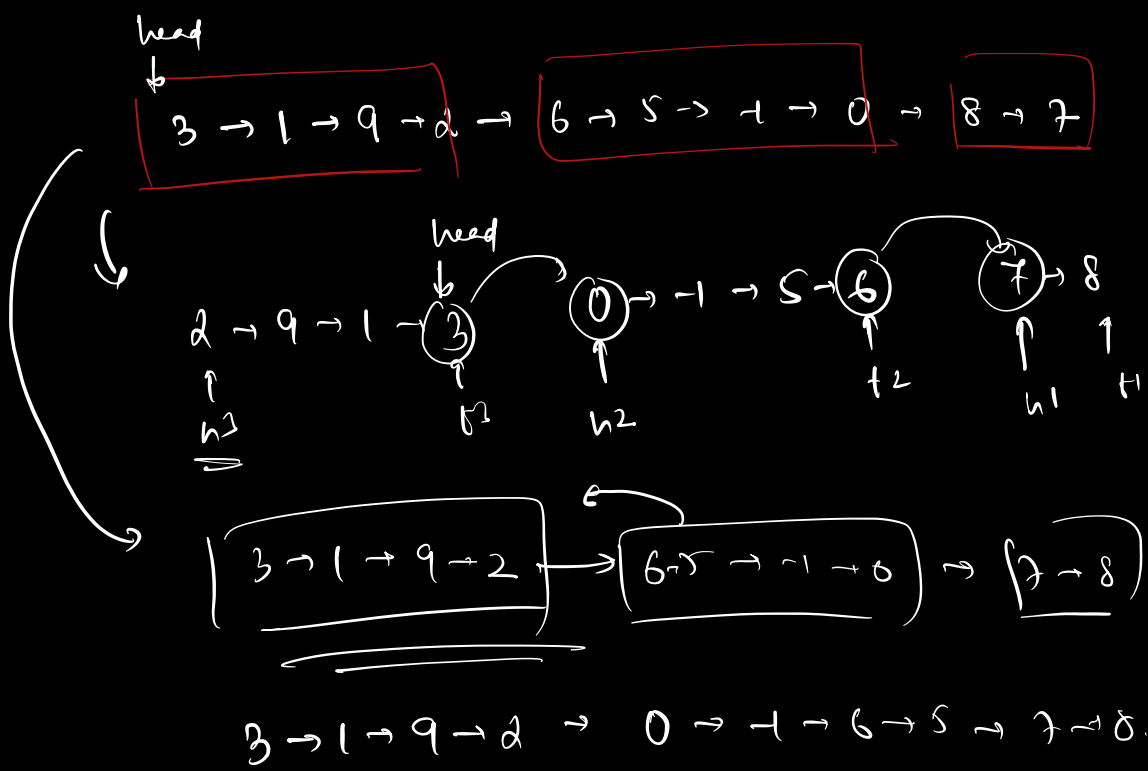


Q Reverse LL in groups of k size

k = 4

3 → 1 → 9 → 2 → 6 → 5 → 4 → 0 → 8 → 7

2 → 9 → 1 → 3 → 0 → 4 → 5 → 6 → 7 → 8



3 → 1 → 9 → 2 → 0 → 4 → 6 → 5 → 7 → 8

1st LL